

暑期妙木山修炼Week1

1.BUUCTF--简单注册器

题目给的是一个apk，说明是安卓逆向

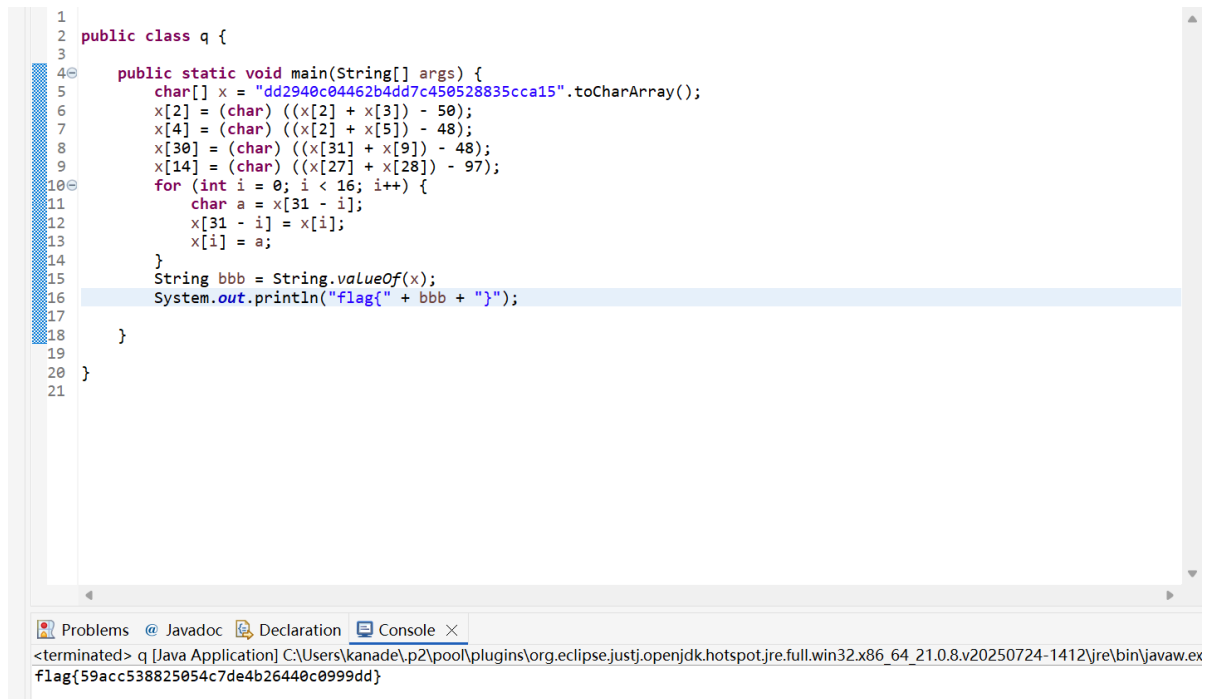
用jadx打开



发现了一个有flag名字的文件，打开发现有mainactivity，直接跟进

里面有很显眼的一段与flag有关的逻辑，猜测经过这个逻辑后得到的就是结果

因此直接把这段代码复制运行一下就行了



另外，此题实际上只要输入满足上面的四个条件，底下的变换便会进行，并输出固定的flag，因此如果可以动态调试或者直接输入的话，编一个满足四个条件的输入，应该就能显示答案。

2.BUUCTF--Younter-Driver

```
1 int __cdecl main_0(int argc, const char **argv, const char **envp)
2 {
3     HANDLE Thread; // [esp+D0h] [ebp-14h]
4     HANDLE hObject; // [esp+DCh] [ebp-8h]
5
6     sub_4110FF();
7     ::hObject = CreateMutexW(0, 0, 0);
8     j_strcpy(Destination, &Source);
9     hObject = CreateThread(0, 0, StartAddress, 0, 0, 0);
10    Thread = CreateThread(0, 0, sub_41119F, 0, 0, 0);
11    CloseHandle(hObject);
12    CloseHandle(Thread);
13    while ( dword_418008 != -1 )
14    ;
15    sub_411190();
16    CloseHandle(::hObject);
17    return 0;
18 }
```

打开主函数发现这是一道多线程题

::hObject = CreateMutexW(0, 0, 0); 创建互斥锁，使只有一个进程能在运行

hObject = CreateThread(0, 0, StartAddress, 0, 0, 0);

Thread = CreateThread(0, 0, sub_41119F, 0, 0, 0); 创建的两个进程

```
1 void __stdcall __noreturn StartAddress_0(int a1)
2 {
3     while ( 1 )
4     {
5         WaitForSingleObject(hObject, 0xFFFFFFFF);
6         if ( dword_418008 > -1 )
7         {
8             sub_41112C((int)&Source, dword_418008);
9             --dword_418008;
10            Sleep(0x64u);
11        }
12        ReleaseMutex(hObject);
13    }
14 }
```

```

1 void __stdcall __noreturn sub_411B10(int a1)
2 {
3     while ( 1 )
4     {
5         WaitForSingleObject(hObject, 0xFFFFFFFF);
6         if ( dword_418008 > -1 )
7         {
8             Sleep(0x64u);
9             --dword_418008;
10        }
11        ReleaseMutex(hObject);
12    }
13 }

```

打开这两个线程可以看到只有一个区别，就是第一个线程多了一个sub41112C函数处理

主函数中dword418008为-1时程序结束，而418008为29，因此要进行三十个线程

Sleep(0x64u) -----让一个线程停止，可以降低cpu占用率

ReleaseMutex(hObject);-----释放互斥锁，让另一个进程运行

补充：一般都是用锁外sleep，即先释放互斥锁，再sleep，这样使得一边在工作的同时另一边休息，使效率翻倍，但是此题不知道为什么用的是锁内sleep，这样非常浪费时间，睡觉的时候另一个线程还在苦苦等待互斥锁释放，不过就结果而言是一样的，都是交替进行。

因此奇数次会受到处理，偶数次保持不变。再来看经过了什么处理。

```

// positive sp value has been detected, the output may be wrong!
char *__cdecl sub_411940(int a1, int a2)
{
    char *result; // eax
    char v3; // [esp+D3h] [ebp-5h]

    v3 = *(_BYTE *)(a2 + a1);
    if ( (v3 < 97 || v3 > 122) && (v3 < 65 || v3 > 90) )
        exit(0);
    if ( v3 < 97 || v3 > 122 )
    {
        result = off_418000[0];
        *(_BYTE *)(a2 + a1) = off_418000[0][*(char *)(a2 + a1) - 38];
    }
    else
    {
        result = off_418000[0];
        *(_BYTE *)(a2 + a1) = off_418000[0][*(char *)(a2 + a1) - 96];
    }
    return result;
}

```

红字是堆栈指针警告，说明参数顺序可能是错误的

下文一直用的a2+a1但是实际上跟a1+a2是一个意思，也就是说看哪个是地址哪个是偏移量就行了，不用严格照这个顺序

a2+a1=a2[a1]或a1[a2]

再来看函数，是一个替换字母表，大写字母换成字母表中的a1[a2]-38位，小写字母换成a1[a2]-96位

```
int sub_411880()
{
    int i; // [esp+D0h] [ebp-8h]

    for ( i = 0; i < 29; ++i )
    {
        if ( Source[i] != off_418004[i] )
            exit(0);
    }
    return printf("\nflag{%s}\n\n", Destination);
}
```

主函数最终进行比较，要使变换后的等于418004

已知字母表和最终结果，直接写脚本反推

```
des='ToiZiZtOrYaToUwPnToBs0a0apsyS'
table='QWERTYUIOPASDFGHJKLZXCVBNMqwertyuiopasdfghjklzxcvbnm'
flag=''
for i in range(29) :
    if i%2==0 :
        flag+=des[i]
    else:
        if ord(des[i])<=ord('Z'):
            flag+=chr(table.index(des[i])+96)
        else:
            flag+=chr(table.index(des[i])+38)

print(flag)
```

得到结果ThisisthreadofwindowshahalsES，由于最终只比较了前29位，因此最后一个不知道，只能爆破尝试

最终结果为ThisisthreadofwindowshahalsESE

3.BUUCTF--findit

题目给了一个apk文件，用jadx打开，定位到mainactivity

```

ed from: classes.dex */
class MainActivity extends ActionBarActivity {
    override // android.support.v7.app.ActionBarActivity, android.support.v4.app.FragmentActivity, android.app.Activity
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        Button btn = (Button) findViewById(R.id.widget3);
        final EditText edit = (EditText) findViewById(R.id.widget2);
        final TextView text = (TextView) findViewById(R.id.widget1);
        final char[] a = {'T', 'h', 'i', 's', 'i', 's', 'T', 'h', 'e', 'F', 'i', 'a', 'g', 'H', 'o', 'm', 'e'};
        final char[] b = {'p', 'v', 'k', 'q', '{', 'm', '1', '6', '4', '6', '7', '5', '2', '6', '2', '0', '3', '3', 'l', '4', 'm', '4', '9', 'l', 'n', 'p', '7', 'p', '9', 'm', 'n', 'k',
        btn.setOnClickListener(new View.OnClickListener() { // from class: com.example.findit.MainActivity.1
            @Override // android.view.View.OnClickListener
            public void onClick(View v) {
                char[] x = new char[17];
                char[] y = new char[38];
                for (int i = 0; i < 17; i++) {
                    if ((a[i] < 'I' && a[i] >= 'A') || (a[i] < 'i' && a[i] >= 'a')) {
                        x[i] = (char) (a[i] + 18);
                    } else if ((a[i] >= 'A' && a[i] <= 'Z') || (a[i] >= 'a' && a[i] <= 'z')) {
                        x[i] = (char) (a[i] - '\b');
                    } else {
                        x[i] = a[i];
                    }
                }
                String m = String.valueOf(x);
                if (m.equals(edit.getText().toString())) {
                    for (int i2 = 0; i2 < 38; i2++) {
                        if ((b[i2] >= 'A' && b[i2] <= 'Z') || (b[i2] >= 'a' && b[i2] <= 'z')) {
                            y[i2] = (char) (b[i2] + 16);
                            if ((y[i2] > 'Z' && y[i2] < 'a') || y[i2] >= 'z') {
                                y[i2] = (char) (y[i2] - 26);
                            }
                        } else {
                            y[i2] = b[i2];
                        }
                    }
                    String n = String.valueOf(y);
                    text.setText(n);
                    return;
                }
                text.setText("答案错了肿么办。。。不给你又不好意思。。。哎呀好纠结啊~~~");
            }
        });
    }
}

```

发现了这题的关键部分，首先对数组x处理，对结果进行比较，通过后再对数组y操作，得到答案

因此这题都不用管x部分，直接复制y部分运行即可，x部分只是验证作用

```
public class q {
```

```
    public static void main(String[] args) {
```

```
        char[] y = new char[38];
```

```
        final char[] a = {'T', 'h', 'i', 's', 'i', 's', 'T', 'h', 'e', 'F', 'i', 'a', 'g', 'H', 'o', 'm', 'e'};
```

```
        final char[] b = {'p', 'v', 'k', 'q', '{', 'm', '1', '6', '4', '6', '7', '5', '2', '6', '2', '0', '3', '3', 'l', '4', 'm', '4', '9',
        'l', 'n', 'p', '7', 'p', '9', 'm', 'n', 'k', '2', '8', 'k', '7', '5', '}'};
```

```
        for (int i2 = 0; i2 < 38; i2++) {
```

```
            if ((b[i2] >= 'A' && b[i2] <= 'Z') || (b[i2] >= 'a' && b[i2] <= 'z')) {
```

```
                y[i2] = (char) (b[i2] + 16);
```

```
                if ((y[i2] > 'Z' && y[i2] < 'a') || y[i2] >= 'z') {
```

```
                    y[i2] = (char) (y[i2] - 26);
```

```
                }
```

```
            } else {
```

```
                y[i2] = b[i2];
```

```
            }
```

```
        }
```

```
        String n = String.valueOf(y);
```

```

        System.out.println(n);
    }

}

```

得到结果flag{c164675262033b4c49bdf7f9cda28a75}

4.BUUCTF--rome

先shift+F12定位到关键部分

```

1 int func()
2 {
3     int result; // eax
4     int v1[4]; // [esp+14h] [ebp-44h]
5     unsigned __int8 v2; // [esp+24h] [ebp-34h] BYTE
6     unsigned __int8 v3; // [esp+25h] [ebp-33h]
7     unsigned __int8 v4; // [esp+26h] [ebp-32h]
8     unsigned __int8 v5; // [esp+27h] [ebp-31h]
9     unsigned __int8 v6; // [esp+28h] [ebp-30h]
10    int v7; // [esp+29h] [ebp-2Fh]
11    int v8; // [esp+2Dh] [ebp-28h]
12    int v9; // [esp+31h] [ebp-27h]
13    int v10; // [esp+35h] [ebp-23h]
14    unsigned __int8 v11; // [esp+39h] [ebp-1Fh]
15    char v12[29]; // [esp+3Bh] [ebp-1Dh] BYTE
16
17    strcpy(v12, "Qsw3sj_1z4_Ujw01");
18    printf("Please input:");
19    scanf("%s", &v2);
20    result = v2;
21    if ( v2 == 'A' )
22    {
23        result = v3;
24        if ( v3 == 'C' )
25        {
26            result = v4;
27            if ( v4 == 'T' )
28            {
29                result = v5;
30                if ( v5 == 'F' )
31                {
32                    result = v6;
33                    if ( v6 == '{' )
34                    {
35                        result = v11;
36                        if ( v11 == '}' )
37                        {
38                            v1[0] = v7;
39                            v1[1] = v8;
40                            v1[2] = v9;
41                            v1[3] = v10;
42                            *(_DWORD *)&v12[17] = 0;
43                            while ( *(int *)&v12[17] <= 15 )
44                            {
45                                *(_DWORD *)&v12[17] += 26;
46                                if ( *((char *)v1 + *(_DWORD *)&v12[17]) > 64 && *((char *)v1 + *(_DWORD *)&v12[17]) <= 90 )
47                                    *((_BYTE *)v1 + *(_DWORD *)&v12[17]) = (*((char *)v1 + *(_DWORD *)&v12[17]) - 51) % 26 + 65;
48                                if ( *((char *)v1 + *(_DWORD *)&v12[17]) > 96 && *((char *)v1 + *(_DWORD *)&v12[17]) <= 122 )
49                                    *((_BYTE *)v1 + *(_DWORD *)&v12[17]) = (*((char *)v1 + *(_DWORD *)&v12[17]) - 79) % 26 + 97;
50                                ++*(_DWORD *)&v12[17];
51                            }
52                            *(_DWORD *)&v12[17] = 0;
53                            while ( *(int *)&v12[17] <= 15 )
54                            {
55                                result = (unsigned __int8)v12[*(_DWORD *)&v12[17]];
56                                if ( *((_BYTE *)v1 + *(_DWORD *)&v12[17]) != (_BYTE)result )
57                                    return result;
58                                ++*(_DWORD *)&v12[17];
59                            }
60                            return printf("You are correct!");
61                        }
62                    }
63                }
64            }
65        }
66    }
67    return result;
68 }

```

这里v12有30个位置，前16个存放字符，后面空余的位置题目使用v12【17】创建了一个4字节指针用来存放数字当索引用，这样不用再创建新变量，节省空间

后面是一个移位处理，大写字母右移了14位，小写字母右移了18位

因此对结果以同样方式分别右移12位，8位即可得到结果

```

s='Qsw3sj_lz4_Ujw@l'
flag=''
for i in range(16):
    if ord(s[i])<91 and ord(s[i])>64:
        flag+=chr((ord(s[i])-53)%26+65)
    elif ord(s[i])>96 and ord(s[i])<123:
        flag+=chr((ord(s[i])-89)%26+97)
    else:
        flag+=s[i]

print(flag)

```

结果flag{Cae3ar_th4_Gre@t}

5.BUUCTF--login

打开文件后发现给的是个网页

ctrl+U查看源码

```

<!DOCTYPE html >
<html>
<head>
<title>FLARE On 2017</title>
</head>
<body>
<input type="text" name="flag" id="flag" value="Enter the flag" />
<input type="button" id="prompt" value="Click to check the flag" />
<script type="text/javascript">
    document.getElementById("prompt").onclick = function () {
        var flag = document.getElementById("flag").value;
        var rotFlag = flag.replace(/([a-zA-Z])/g, function(c) {return String.fromCharCode((c <= "Z" ? 90 : 122) >= (c = c.charCodeAt(0) + 13) ? c : c - 26);});
        if ("PyvragFvarYbtvafNerRafI@syner-ba.pbz" == rotFlag) {
            alert("Correct flag!");
        } else {
            alert("Incorrect flag. rot again");
        }
    }
</script>
</body>
</html>

```

可以大致看出是rot13相关

直接rot13解密得到ClientSideLoginsAreEasy@flare-on.com

6.BUUCTF--pyre

给了一个pyc文件，用pycdc反编译得到

```

print 'Welcome to Re World!'
print 'Your input1 is your flag~'
l = len(input1)
for i in range(l):
    num = ((input1[i] + i) % 128 + 128) % 128
    code += num

for i in range(l - 1):
    code[i] = code[i] ^ code[i + 1]

```

print code

code = [

'\x1f',

'\x12',

'\x1d',

(',

'0',

'4',

'\x01',

'\x06',

'\x14',

'4',

','

'\x1b',

'U',

'?',

'o',

'6',

'*',

':',

'\x01',

'D',

','

'%',

'\x13']

直接写脚本反推即可

```
code = [
```

```
'\x1f',
```

```
'\x12',
```

```
'\x1d',
```

```
('(',
```

```
'0',
```

```
'4',
```

```
'\x01',
```

```
'\x06',
```

```
'\x14',
```

```
'4',
```

```
','
```

```
'\x1b',
```

```
'U',
```

```
'?',
```

```
'o',
```

```
'6',
```

```
'*',
```

```
':',
```

```
'\x01',
```

```
'D',
```

```
','
```

```
'%',
```

```
'\x13']
```

```
flag=''
```



```

for i in range (22):
    code[21-i]=chr(ord(code[22-i])^ord(code[21-i]))
for i in range (23):
    flag+=chr((ord(code[i])-i)%128)
print(flag)

```

结果GWHT{Just_Re_1s_Ha66y!}

7.BUUCTF--Maze

一道迷宫题，打开发现关键部分反编译不了，因为有花指令，要去花后才能反编译，但是实际上根据字符串中的迷宫内容也能直接偷鸡做，因为这题迷宫没有放在动态下。

8.BUUCTF--xxor

打开主函数

```

1 __int64 __fastcall main(int a1, char **a2, char **a3)
2 {
3     int i; // [rsp+8h] [rbp-68h]
4     int j; // [rsp+Ch] [rbp-64h]
5     __int64 v6[6]; // [rsp+10h] [rbp-60h] BYREF
6     __int64 v7[6]; // [rsp+40h] [rbp-30h] BYREF
7
8     v7[5] = __readfsqword(0x28u);
9     puts("Let us play a game?");
10    puts("you have six chances to input");
11    puts("Come on!");
12    memset(v6, 0, 40);
13    for ( i = 0; i <= 5; ++i )
14    {
15        printf("%s", "input: ");
16        __isoc99_scanf("%d", (char *)v6 + 4 * i);
17    }
18    memset(v7, 0, 40);
19    for ( j = 0; j <= 2; ++j )
20    {
21        dword_601078 = v6[j];
22        dword_60107C = HIDWORD(v6[j]);
23        sub_400686((unsigned int *)&dword_601078, dword_601060);
24        LODWORD(v7[j]) = dword_601078;
25        HIDWORD(v7[j]) = dword_60107C;
26    }
27    if ( (unsigned int)sub_400770(v7) != 1 )
28    {
29        puts("NO NO NO~ ");
30        exit(0);
31    }
32    puts("Congretulation!\n");
33    puts("You seccess half\n");
34    puts("Do not forget to change input to hex and combine~\n");
35    puts("ByeBye");
36    return 0LL;
37 }

```

这题的逻辑是，输入六个数往v6里存，v6是一个64位整型数组，把它转成char指针使其地址一个一个字节移动，每次移动4个字节，也就是说每次都是填一个数据的一半，即32位，这也正好与下文的高低位对应。

中间一段是对输入的数进行处理，先不看，看最后的比较函数

```

__int64 __fastcall sub_400770(_DWORD *a1)
{
    if ( a1[2] - a1[3] == 0x84A236FFLL
        && a1[3] + a1[4] == 0xFA6CB703LL
        && a1[2] - a1[4] == 0x42D731A8LL
        && *a1 == 0xDF48EF7E
        && a1[5] == 0x84F30420
        && a1[1] == 0x20CAACF4 )
    {
        puts("good!");
        return 1LL;
    }
    else
    {
        puts("Wrong!");
        return 0LL;
    }
}

```

根据这个可以得到变换后的输入，就是一个解方程，我找了一个别人写好的z3脚本，注意数都要换成16进制

```

from z3 import *

a0, a1, a2, a3, a4, a5 = Ints('a0 a1 a2 a3 a4 a5')
s = Solver()
s.add(a2 - a3 == 0x84A236FF)
s.add(a3 + a4 == 0xFA6CB703)
s.add(a2 - a4 == 0x42D731A8)
s.add(a0 == 0xDF48EF7E)
s.add(a5 == 0x84F30420)
s.add(a1 == 0x20CAACF4)
if s.check() == sat:
    print(s.model())

```

结果[a4 = 2652626477,
a0 = 3746099070,
a5 = 2230518816,
a2 = 3774025685,
a1 = 550153460,
a3 = 1548802262]

有了最终结果再根据中间的变换反推输入

```

1 __int64 __fastcall sub_400686(unsigned int *a1, _DWORD *a2)
2 {
3     __int64 result; // rax
4     unsigned int v3; // [rsp+1Ch] [rbp-24h]
5     unsigned int v4; // [rsp+20h] [rbp-20h]
6     int v5; // [rsp+24h] [rbp-1Ch]
7     unsigned int i; // [rsp+28h] [rbp-18h]
8
9     v3 = *a1;
10    v4 = a1[1];
11    v5 = 0;
12    for ( i = 0; i <= 63; ++i )
13    {
14        v5 += 1166789954;
15        v3 += (v4 + v5 + 11) ^ ((v4 << 6) + *a2) ^ ((v4 >> 9) + a2[1]) ^ 0x20;
16        v4 += (v3 + v5 + 20) ^ ((v3 << 6) + a2[2]) ^ ((v3 >> 9) + a2[3]) ^ 0x10;
17    }
18    *a1 = v3;
19    result = v4;
20    a1[1] = v4;
21    return result;
22 }

```

很明显是个tea算法，写脚本

```

#include <stdio.h>
int main()
{
    int a1[6] = { 3746099070,550153460,3774025685,1548802262,2652626477,2230518816 };
    unsigned int a2[4] = { 2,2,3,4 };
    unsigned int v3;
    unsigned int v4;
    int v5;
    for (unsigned int j = 0; j < 5; j += 2)
    {
        v3 = a1[j];
        v4 = a1[j + 1];
        v5 = 1166789954 * 64;
        for (unsigned int i = 0; i < 64; i++)
        {
            v4 -= (v3 + v5 + 20) ^ ((v3 << 6) + a2[2]) ^ ((v3 >> 9) + a2[3]) ^ 0x10;
            v3 -= (v4 + v5 + 11) ^ ((v4 << 6) + *a2) ^ ((v4 >> 9) + a2[1]) ^ 0x20;
            v5 -= 1166789954;
        }
        a1[j] = v3;
        a1[j + 1] = v4;
    } //小端序
    for (unsigned int i = 0; i < 6; i++)
        printf("%c%c%c", ((char)&a1[i] + 2), ((char)&a1[i] + 1), (char)&a1[i]);
}

```

这里省事用的别人的，不过我没看懂为什么最后打印只打印三个字节，怎么知道输入有没有填满4字节的，但是这题就用4字节打印应该也没有影响，因为空字符不会显示

9.BUUCTF--usualcrypt

```
1 int __cdecl main(int argc, const char **argv, const char **envp)
2 {
3     int v3; // esi
4     int v5[3]; // [esp+8h] [ebp-74h] BYREF
5     __int16 v6; // [esp+14h] [ebp-68h]
6     char v7; // [esp+16h] [ebp-66h]
7     char v8[100]; // [esp+18h] [ebp-64h] BYREF
8
9     sub_403CF8((int)&unk_40E140);
10    scanf("%s", v8);
11    memset(v5, 0, sizeof(v5));
12    v6 = 0;
13    v7 = 0;
14    sub_401080((int)v8, strlen(v8), (int)v5);
15    v3 = 0;
16    while ( *((_BYTE *)v5 + v3) == byte_40E0E4[v3] )
17    {
18        if ( ++v3 > strlen((const char *)v5) )
19            goto LABEL_6;
20    }
21    sub_403CF8((int)aError);
22 LABEL_6:
23    if ( v3 - 1 == strlen(byte_40E0E4) )
24        return sub_403CF8((int)aAreYouHappyYes);
25    else
26        return sub_403CF8((int)aAreYouHappyNo);
27 }
```

先看主函数，403CF8很明显是个print函数，然后先对输入进行函数操作，后面都是将结果与一个已知量进行比较，也就是根据结果反推输入的题

```
1 int __cdecl sub_401080(int a1, int a2, int a3)
2 {
3     int v3; // edi
4     int v4; // esi
5     int v5; // edx
6     int v6; // eax
7     int v7; // ecx
8     int v8; // esi
9     int v9; // esi
10    int v10; // esi
11    int v11; // esi
12    BYTE *v12; // ecx
13    int v13; // esi
14    int v15; // [esp+18h] [ebp+8h]
15
16    v3 = 0;
17    v4 = 0;
18    sub_401000();
19    v5 = a2 % 3;
20    v6 = a1;
21    v7 = a2 - a2 % 3;
22    v15 = a2 % 3;
23    if ( v7 > 0 )
24    {
25        do
26        {
27            LOBYTE(v5) = *((_BYTE *) (a1 + v3));
28            v3 += 3;
29            v8 = v4 + 1;
30            *((_BYTE *) (v8 + a3 - 1)) = byte_40E0A0[(v5 >> 2) & 0x3F];
31            *((_BYTE *) (++v8 + a3 - 1)) = byte_40E0A0[16 * *((_BYTE *) (a1 + v3 - 3) & 3)
32                + (((int) *((unsigned __int8 *) (a1 + v3 - 2) >> 4) & 0xF)];
33            *((_BYTE *) (++v8 + a3 - 1)) = byte_40E0A0[4 * *((_BYTE *) (a1 + v3 - 2) & 0xF)
34                + (((int) *((unsigned __int8 *) (a1 + v3 - 1) >> 6) & 3)];
35            v5 = *((_BYTE *) (a1 + v3 - 1) & 0x3F);
36            v4 = v8 + 1;
37            *((_BYTE *) (v4 + a3 - 1)) = byte_40E0A0[v5];
38        }
39        while ( v3 < v7 );
40        v5 = v15;
41    }
42    if ( v5 == 1 )
43    {
44        LOBYTE(v7) = *((_BYTE *) (v3 + a1));
45    }
46 }
```

```

v9 = v4 + 1;
*(_BYTE *)(v9 + a3 - 1) = byte_40E0A0[(v7 >> 2) & 0x3F];
v10 = v9 + 1;
*(_BYTE *)(v10 + a3 - 1) = byte_40E0A0[16 * (*(_BYTE *)(v3 + a1) & 3)];
*(_BYTE *)(v10 + a3) = 61;
LABEL_8:
v13 = v10 + 1;
*(_BYTE *)(v13 + a3) = 61;
v4 = v13 + 1;
goto LABEL_9;
}
if ( v5 == 2 )
{
    v11 = v4 + 1;
    *(_BYTE *)(v11 + a3 - 1) = byte_40E0A0[((int)*(unsigned __int8 *)(v3 + a1) >> 2) & 0x3F];
    v12 = (_BYTE *)(v3 + a1 + 1);
    LOBYTE(v6) = *v12;
    v10 = v11 + 1;
    *(_BYTE *)(v10 + a3 - 1) = byte_40E0A0[16 * (*(_BYTE *)(v3 + a1) & 3) + ((v6 >> 4) & 0xF)];
    *(_BYTE *)(v10 + a3) = byte_40E0A0[4 * (*v12 & 0xF)];
    goto LABEL_8;
}
LABEL_9:
*(_BYTE *)(v4 + a3) = 0;
return sub_401030(a3);
}

```

打开加密函数，可以看出是一个base64加密（分了三组，三字节转四字节，出现0x3F, 61 (=)），但是要注意的是开头用了一个sub401000，结尾用了一个sub401030，要看这两个函数又进行了什么操作

第一个是替换了一下加密表

```

1 int sub_401000()
2 {
3     int result; // eax
4     char v1; // cl
5
6     for ( result = 6; result < 15; ++result )
7     {
8         v1 = byte_40E0AA[result];
9         byte_40E0AA[result] = byte_40E0A0[result];
10        byte_40E0A0[result] = v1;
11    }
12    return result;
13 }

```

脚本

```

#include <stdio.h>
int main()
{
    char b[]="ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/";
    char c;
    int i=6;
    printf("sss\n");
    do
    {
        c=b[i+10];
        b[i+10]=b[i];
        b[i++]=c;
    }
}

```

```

}
while(i<15);

printf("%s",b);

}

```

得到码表: ABCDEFQRSTUVWXYPGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/-

```

int __cdecl sub_401030(const char *a1)
{
    __int64 v1; // rax
    char v2; // al

    v1 = 0i64;
    if ( strlen(a1) )
    {
        do
        {
            v2 = a1[HIDWORD(v1)];
            if ( v2 < 97 || v2 > 122 )
            {
                if ( v2 < 65 || v2 > 90 )
                    goto LABEL_9;
                LOBYTE(v1) = v2 + 32;
            }
            else
            {
                LOBYTE(v1) = v2 - 32;
            }
            a1[HIDWORD(v1)] = v1;
        LABEL_9:
            LODWORD(v1) = 0;
            ++HIDWORD(v1);
        }
        while ( HIDWORD(v1) < strlen(a1) );
    }
    return v1;
}

```

最后返回时的函数很明显是一个大小写互换

因此直接把最终结果换大小写,再用新的base表解码即可

flag{bAse64_h2s_a_Surprise}

10.BUUCTF--helloworldgo

这题主要是了解一下go语言，go语言写的程序一般比较大，伪代码也特别多和乱，这题查找字符串flag即可

11.BUUCTF--ignite me

```
1 void __noreturn start()
2 {
3     DWORD NumberOfBytesWritten; // [esp+0h] [ebp-4h] BYREF
4
5     NumberOfBytesWritten = 0;
6     hFile = GetStdHandle(0xFFFFFFFF6);
7     dword_403074 = GetStdHandle(0xFFFFFFFF5);
8     WriteFile(dword_403074, aG1v3M3T3hF14g, 0x13u, &NumberOfBytesWritten, 0);
9     sub_4010F0();
10    if ( sub_401050() )
11        WriteFile(dword_403074, aG00dJ0b, 0xAu, &NumberOfBytesWritten, 0);
12    else
13        WriteFile(dword_403074, aN0tT00H0tRWe7r, 0x24u, &NumberOfBytesWritten, 0);
14    ExitProcess(0);
15 }
```

看主函数发现关键在于sub401050这个函数上，只要这个函数对了就成功

```
1 int sub_401050()
2 {
3     int v1; // [esp+0h] [ebp-Ch]
4     int i; // [esp+4h] [ebp-8h]
5     unsigned int j; // [esp+4h] [ebp-8h]
6     char v4; // [esp+Bh] [ebp-1h]
7
8     v1 = sub_401020((int)byte_403078);
9     v4 = sub_401000();
10    for ( i = v1 - 1; i >= 0; --i )
11    {
12        byte_403180[i] = v4 ^ byte_403078[i];
13        v4 = byte_403078[i];
14    }
15    for ( j = 0; j < 39; ++j )
16    {
17        if ( byte_403180[j] != (unsigned __int8)byte_403000[j] )
18            return 0;
19    }
20    return 1;
21 }
```

401020相当于strlen，v4是个值，这题的逻辑就是逆序后一位与现在这位异或，第一位与v4异或，最后比较

因此只要知道v4就能写脚本反推了，v4打开函数后发现是用会被写的移位循环，所以采用动调在这句话后下断点来看v4的值

```
Stack[0000817C]:001AFF63 db 0
Stack[0000817C]:001AFF64 db 13h
Stack[0000817C]:001AFF65 db 0
Stack[0000817C]:001AFF66 db 0
Stack[0000817C]:001AFF67 db 4
Stack[0000817C]:001AFF68 db 74h ; t
Stack[0000817C]:001AFF69 db 0FFh
Stack[0000817C]:001AFF6A db 1Ah
Stack[0000817C]:001AFF6B db 0
Stack[0000817C]:001AFF6C db 0F9h
Stack[0000817C]:001AFF6D db 11h
Stack[0000817C]:001AFF6E db 40h ; @
Stack[0000817C]:001AFF6F db 0
Stack[0000817C]:001AFF70 db 13h
Stack[0000817C]:001AFF71 db 0
Stack[0000817C]:001AFF72 db 0
Stack[0000817C]:001AFF73 db 0
Stack[0000817C]:001AFF74 db 84h
Stack[0000817C]:001AFF75 db 0FFh
Stack[0000817C]:001AFF76 db 1Ah
Stack[0000817C]:001AFF77 db 0
Stack[0000817C]:001AFF78 db 49h ; I
Stack[0000817C]:001AFF79 db 5Dh ; ]
Stack[0000817C]:001AFF7A db 0DEh
Stack[0000817C]:001AFF7B db 75h ; u
Stack[0000817C]:001AFF7C db 0
Stack[0000817C]:001AFF7D db 0F0h
Stack[0000817C]:001AFF7E db 37h ; 7
Stack[0000817C]:001AFF7F db 0
UNKNOWN 001AFF67: Stack[0000817C:001AFF67] (Synchronized with EIP)
```

得到v4是4后就可以写脚本了

```
a=[0x0D, 0x26, 0x49, 0x45, 0x2A, 0x17, 0x78, 0x44, 0x2B, 0x6C,
    0x5D, 0x5E, 0x45, 0x12, 0x2F, 0x17, 0x2B, 0x44, 0x6F, 0x6E,
    0x56, 0x09, 0x5F, 0x45, 0x47, 0x73, 0x26, 0x0A, 0x0D, 0x13,
    0x17, 0x48, 0x42, 0x01, 0x40, 0x4D, 0x0C, 0x02, 0x69, 0x00]
v=4
flag=''
for i in range(len(a)-1,-1,-1):
    flag+=chr(a[i]^v)
    v=a[i]^v
print(flag[::-1])
```

答案flag{[R_y0u_H0t_3n0ugH_t0_1gn1t3@flare-on.com](https://www.flare-on.com)}

12.BUUCTF--crackRTF

```
memset(Destination, 0, sizeof(Destination));
memset(String1, 0, sizeof(String1));
v7 = 0;
printf("pls input the first passwd(1): ");
scanf("%s", Destination);
if ( strlen(Destination) != 6 )
{
    printf("Must be 6 characters!\n");
    ExitProcess(0);
}
v7 = atoi(Destination);
if ( v7 < 100000 )
    ExitProcess(0);
strcat(Destination, "@DBApp");
v3 = strlen(Destination);
sub_40100A((BYTE *)Destination, v3, String1);
if ( !_strcmpi(String1, "6E32D0943418C2C33385BC35A1470250DD8923A9") )
{
    printf("continue...\n\n");
    printf("pls input the first passwd(2): ");
    memset(Str, 0, sizeof(Str));
    scanf("%s", Str);
    if ( strlen(Str) != 6 )
    {
        printf("Must be 6 characters!\n");
        ExitProcess(0);
    }
    strcat(Str, Destination);
    memset(String1, 0, sizeof(String1));
    v4 = strlen(Str);
    sub_401019((BYTE *)Str, v4, String1);
    if ( !_strcmpi("27019e688a4e62a649fd99cadaafdb4e", String1) )
    {
        if ( !(unsigned __int8)sub_40100F(Str) )
        {
            printf("Error!!\n");
            ExitProcess(0);
        }
        printf("bye ~~\n");
    }
}
return 0;
}
```

打开后顺着分析，首先必须输入长度必须为6，然后把输入拼接一个字符串，经过sub40100A后比较看看这个加密函数是什么

```

1 int __cdecl sub_401230(BYTE *pbData, DWORD dwDataLen, LPSTR lpString1)
2 {
3     DWORD i; // [esp+4Ch] [ebp-28h]
4     CHAR String2[4]; // [esp+50h] [ebp-24h] BYREF
5     BYTE v6[20]; // [esp+54h] [ebp-20h] BYREF
6     DWORD pdwDataLen; // [esp+68h] [ebp-Ch] BYREF
7     HCRYPTHASH phHash; // [esp+6Ch] [ebp-8h] BYREF
8     HCRYPTPROV phProv; // [esp+70h] [ebp-4h] BYREF
9
10    if ( !CryptAcquireContextA(&phProv, 0, 0, 1u, 0xF0000000) )
11        return 0;
12    if ( CryptCreateHash(phProv, 0x8004u, 0, 0, &phHash) )
13    {
14        if ( CryptHashData(phHash, pbData, dwDataLen, 0) )
15        {
16            CryptGetHashParam(phHash, 2u, v6, &pdwDataLen, 0);
17            *lpString1 = 0;
18            for ( i = 0; i < pdwDataLen; ++i )
19            {
20                wsprintfA(String2, "%02X", v6[i]);
21                lstrcatA(lpString1, String2);
22            }
23            CryptDestroyHash(phHash);
24            CryptReleaseContext(phProv, 0);
25            return 1;
26        }
27        else
28        {
29            CryptDestroyHash(phHash);
30            CryptReleaseContext(phProv, 0);
31            return 0;
32        }
33    }
34    else
35    {
36        CryptReleaseContext(phProv, 0);
37        return 0;
38    }
39 }

```

这里可以看到是跟hash有关的加密，但是我不知道具体是哪种，上网查找后发现0x8004u通常在CryptCreateHash函数中出现且这个标志着sha1，0x8003u是md5

```

from hashlib import *
enc = '6e32d0943418c2c33385bc35a1470250dd8923a9'
flag1 = ''
for i in range(10000,999999):
    if sha1((str(i) + "@DBApp").encode("utf8")).hexdigest() == enc:
        flag1 += str(i) + "@DBApp"
print(flag1)

```

这里也是借鉴的别人写的脚本，以后遇到hash可以照葫芦画瓢

得到123321

第二次几乎跟第一次一样，只是换成了0x8003u，也就是md5

由于第一次是全部已知转成数字了，第二次不知道字符串是什么组成，爆破数字得不到结果，用脚本不现实，但是在线网站是可以直接爆破的

直接输入两次密码就可得到一个rtf文件，里面是flag

但是这题并非是想让我们这样做，底下还有一个函数

```
char __cdecl sub_4014D0(LPCSTR lpString)
{
    LPCVOID lpBuffer; // [esp+50h] [ebp-1Ch]
    DWORD NumberOfBytesWritten; // [esp+58h] [ebp-14h] BYREF
    DWORD nNumberOfBytesToWrite; // [esp+5Ch] [ebp-10h]
    HGLOBAL hResData; // [esp+60h] [ebp-Ch]
    HRSRC hResInfo; // [esp+64h] [ebp-8h]
    HANDLE hFile; // [esp+68h] [ebp-4h]

    hFile = 0;
    hResData = 0;
    nNumberOfBytesToWrite = 0;
    NumberOfBytesWritten = 0;
    hResInfo = FindResourceA(0, (LPCSTR)0x65, "AAA");
    if ( !hResInfo )
        return 0;
    nNumberOfBytesToWrite = SizeofResource(0, hResInfo);
    hResData = LoadResource(0, hResInfo);
    if ( !hResData )
        return 0;
    lpBuffer = LockResource(hResData);
    sub_401005(lpString, (int)lpBuffer, nNumberOfBytesToWrite);
    hFile = CreateFileA("dbapp.rtf", 0x10000000u, 0, 0, 2u, 0x80u, 0);
    if ( hFile == (HANDLE)-1 )
        return 0;
    if ( !WriteFile(hFile, lpBuffer, nNumberOfBytesToWrite, &NumberOfBytesWritten, 0) )
        return 0;
    CloseHandle(hFile);
    return 1;
}
```

当然也是啥都看不懂，上网学习了一下别人的wp，才知道这是要用一个叫resourcehacker的软件（查找资源）

```
unsigned int __cdecl sub_401420(LPCSTR lpString, int a2, unsigned int a3)
{
    unsigned int result; // eax
    unsigned int i; // [esp+4Ch] [ebp-Ch]
    unsigned int v5; // [esp+54h] [ebp-4h]

    v5 = strlenA(lpString);
    for ( i = 0; ; ++i )
    {
        result = i;
        if ( i >= a3 )
            break;
        *(_BYTE *)(i + a2) ^= lpString[i % v5];
    }
    return result;
}
```

里面还有一个函数，大致意思就是用资源的前六位和密码前六位异或，得到rtf文件的文件头，于是我们反过来异或就得到第二次的密码了

这题实在是有点.....不知道这个查找资源究竟有啥用，还专门要有这个软件，有网的话还是在线爆破吧...

13.BUUCTF--rsa

名称	修改日期	类型	大小
 flag	2026/7/26 23:49	Wireshark capture f...	1 KB
 pub.key	2026/7/26 23:49	KEY 文件	1 KB

题目给了一个flag流量包和公钥文件，首先需要公钥解析，直接用脚本

```
from Crypto.PublicKey import RSA
with open('pub.key','rb') as f:
    key = RSA.import_key(f.read())
    print('n = %d' % key.n)
    print('e = %d' % key.e)
```

得到n和e后可以得到p,q,phi,此题密文是这个流量包，对于这种形式的密文也直接上脚本就行

```
import rsa

e= 65537
n= 86934482296048119190666062003494800588905656017203025617216654058378322103517
p= 285960468890451637935629440372639283459
q= 304008741604601924494328155975272418463
d= 81176168860169991027846870170527607562179635470395365333547868786951080991441

key = rsa.PrivateKey(n,e,d,q,p)          #在pkcs标准中,pkcs#1规定,私钥包含(n,e,d,p,q)

with open(r"D:\re题目
\b02a8dfce5ad8cfdfa0a94c826aab55f50162d0b54e541fe2a404934b76ce276\output\flag.enc
","rb") as f:  #以二进制读模式,读取密文
    f = f.read()
    print(rsa.decrypt(f,key))            # f:公钥加密结果  key:私钥
```

得到b'flag{decrypt_256}\n'

14.BUUCTF--overlong

```
1 int __stdcall start(int a1, int a2, int a3, int a4)
2 {
3     CHAR Text[128]; // [esp+0h] [ebp-84h] BYREF
4     int v6; // [esp+80h] [ebp-4h]
5
6     v6 = sub_401160(Text, &unk_402008, 28);
7     Text[v6] = 0;
8     MessageBoxA(0, Text, Caption, 0);
9     return 0;
10 }
```

```

1 unsigned int __cdecl sub_401160(char *a1, int a2, unsigned int a3)
2 {
3     unsigned int i; // [esp+4h] [ebp-4h]
4
5     for ( i = 0; i < a3; ++i )
6     {
7         a2 += sub_401000(a1, a2);
8         if ( !*a1++ )
9             break;
10    }
11    return i;
12 }

```

```

1 int __cdecl sub_401000(_BYTE *a1, char *a2)
2 {
3     int v3; // [esp+0h] [ebp-8h]
4     char v4; // [esp+4h] [ebp-4h]
5
6     if ( (int)(unsigned __int8)*a2 >> 3 == 30 )
7     {
8         v4 = a2[3] & 0x3F | ((a2[2] & 0x3F) << 6);
9         v3 = 4;
10    }
11    else if ( (int)(unsigned __int8)*a2 >> 4 == 14 )
12    {
13        v4 = a2[2] & 0x3F | ((a2[1] & 0x3F) << 6);
14        v3 = 3;
15    }
16    else if ( (int)(unsigned __int8)*a2 >> 5 == 6 )
17    {
18        v4 = a2[1] & 0x3F | ((*a2 & 0x1F) << 6);
19        v3 = 2;
20    }
21    else
22    {
23        v4 = *a2;
24        v3 = 1;
25    }
26    *a1 = v4;
27    return v3;
28 }

```

程序的主要逻辑和函数如上，就是对原始数据进行加密然后显示，但是原始数据是很长的，这里只用到了28位，从输出结果可以看到程序应该是没输出完的，因此让它全部用上就行。

第一种方法就是传统的复现流程

```

char unk_402008[] =
{
    224, 129, 137, 192, 160, 193, 174, 224, 129, 165,
    193, 182, 240, 128, 129, 165, 224, 129, 178, 240,
    128, 128, 160, 224, 129, 162, 114, 111, 193, 171,
    101, 224, 128, 160, 224, 129, 180, 224, 129, 168,
    193, 165, 32, 193, 165, 224, 129, 174, 99, 193,
    175, 224, 129, 164, 240, 128, 129, 169, 110, 193,
    167, 192, 186, 32, 73, 240, 128, 129, 159, 193,
    161, 193, 159, 193, 141, 224, 129, 159, 193, 180,
    240, 128, 129, 159, 240, 128, 129, 168, 193, 159,
    240, 128, 129, 165, 224, 129, 159, 193, 165, 224,
    129, 159, 240, 128, 129, 174, 193, 159, 240, 128,
    129, 131, 193, 159, 224, 129, 175, 224, 129, 159,
    193, 132, 95, 224, 129, 169, 240, 128, 129, 159,
    110, 224, 129, 159, 224, 129, 167, 224, 129, 128,
    240, 128, 129, 166, 240, 128, 129, 172, 224, 129,
    161, 193, 178, 193, 165, 240, 128, 128, 173, 240,
    128, 129, 175, 110, 192, 174, 240, 128, 129, 163,
    111, 240, 128, 129, 173
};
int __cdecl sub_401000(char* a1, char* a2)
{
    int v3; // [esp+0h] [ebp-8h]
    char v4; // [esp+4h] [ebp-4h]

```

```

    if ((int)(unsigned __int8)*a2 >> 3 == 30)
    {
        v4 = a2[3] & 0x3F | ((a2[2] & 0x3F) << 6);
        v3 = 4;
    }
    else if ((int)(unsigned __int8)*a2 >> 4 == 14)
    {
        v4 = a2[2] & 0x3F | ((a2[1] & 0x3F) << 6);
        v3 = 3;
    }
    else if ((int)(unsigned __int8)*a2 >> 5 == 6)
    {
        v4 = a2[1] & 0x3F | ((*a2 & 0x1F) << 6);
        v3 = 2;
    }
    else
    {
        v4 = *a2;
        v3 = 1;
    }
    *a1 = v4;
    return v3;

```

```

}
unsigned int __cdecl sub_401160(char* a1,char* a2, unsigned int a3)
{
    unsigned int i; // [esp+4h] [ebp-4h]

```

```

    for (i = 0; i < a3; ++i)
    {
        a2 += sub_401000(a1, a2);
        if (!*a1++)
            break;
    }
    return i;

```

```

}
int main()
{
    char Text[128]; // [esp+0h] [ebp-84h] BYREF
    int v6; // [esp+80h] [ebp-4h]

```

```

    v6 = sub_401160(Text, unk_402008, 28);
    Text[v6] = 0;
    int i;
    for (i = 0; i < 28; ++i)
    {
        printf("%c", Text[i]);
    }
    return 0;

```

```

}

```

这里最好就是用c语言，用python转换起来就复杂了，把代码中的28改大就行

但是这题真正要学的不是这个，而是通过动态调试修改程序

000811B0	CC	int3	
000811B6	CC	int3	
000811BF	CC	int3	
000811C0	55	push ebp	OptionalHeader.AddressOfEntryPoint
000811C1	8BEC	mov ebp,esp	
000811C3	81EC 84000000	sub esp,84	
000811C9	6A 1C	push 1C	
000811CB	68 0820D800	push overlong.D82008	
000811D0	8B85 7CFFFFFF	lea eax,dword ptr ss:[ebp-84]	
000811D6	50	push eax	
000811D7	E8 84FFFFFF	call overlong.D81160	
000811DC	83C4 0C	add esp,C	
000811DF	8945 FC	mov dword ptr ss:[ebp-4],eax	
000811E2	8B4D FC	mov ecx,dword ptr ss:[ebp-4]	
000811E5	C6840D 7CFFFFFF 00	mov byte ptr ds:[ecx-84],0	
000811ED	6A 00	push 0	
000811EF	68 0030D800	push overlong.D83000	DB3000: "Output"
000811F4	8D95 7CFFFFFF	lea edx,dword ptr ss:[ebp-84]	
000811FA	52	push edx	
000811FB	6A 00	push 0	
000811FD	FF15 0020D800	call dword ptr ds:[<MessageBoxA>]	
00081203	33C0	xor eax,eax	
00081205	8BE5	mov esp,ebp	
00081207	5D	pop ebp	
00081208	C2 1000	ret 10	
0008120B <JMP	FF25 0020D800	jmp dword ptr ds:[<MessageBoxA>]	JMP.<MessageBoxA>
00081211	0000	add byte ptr ds:[eax],al	
00081213	0000	add byte ptr ds:[eax],al	
00081215	0000	add byte ptr ds:[eax],al	
00081217	0000	add byte ptr ds:[eax],al	
00081219	0000	add byte ptr ds:[eax],al	
0008121B	0000	add byte ptr ds:[eax],al	
0008121D	0000	add byte ptr ds:[eax],al	
0008121F	0000	add byte ptr ds:[eax],al	
00081221	0000	add byte ptr ds:[eax],al	
00081223	0000	add byte ptr ds:[eax],al	
00081225	0000	add byte ptr ds:[eax],al	
00081227	0000	add byte ptr ds:[eax],al	
00081229	0000	add byte ptr ds:[eax],al	
0008122B	0000	add byte ptr ds:[eax],al	
0008122D	0000	add byte ptr ds:[eax],al	
0008122F	0000	add byte ptr ds:[eax],al	
00081231	0000	add byte ptr ds:[eax],al	
00081233	0000	add byte ptr ds:[eax],al	
00081235	0000	add byte ptr ds:[eax],al	

通过查找找到1C，发现了一个push1C，就是说传入28，把28修改成80，在运行就能得到最终结果

Output



I never broke the encoding: I_a_M_t_h_e_e_n_C_o_D_i_n_g@flare-on.com

确定

15.BUUCTF--re

这题考的就是z3约束，正好好好学习一下怎么写


```

1  _BOOL8 __fastcall sub_4009AE(char *a1)
2  {
3      if ( 1629056 * *a1 != 166163712 )
4          return 0LL;
5      if ( 6771600 * a1[1] != 731332800 )
6          return 0LL;
7      if ( 3682944 * a1[2] != 357245568 )
8          return 0LL;
9      if ( 10431000 * a1[3] != 1074393000 )
10         return 0LL;
11     if ( 3977328 * a1[4] != 489211344 )
12         return 0LL;
13     if ( 5138336 * a1[5] != 518971936 )
14         return 0LL;
15     if ( 7532250 * a1[7] != 406741500 )
16         return 0LL;
17     if ( 5551632 * a1[8] != 294236496 )
18         return 0LL;
19     if ( 3409728 * a1[9] != 177305856 )
20         return 0LL;
21     if ( 13013670 * a1[10] != 650683500 )
22         return 0LL;
23     if ( 6088797 * a1[11] != 298351053 )
24         return 0LL;
25     if ( 7884663 * a1[12] != 386348487 )
26         return 0LL;
27     if ( 8944053 * a1[13] != 438258597 )
28         return 0LL;
29     if ( 5198490 * a1[14] != 249527520 )
30         return 0LL;
31     if ( 4544518 * a1[15] != 445362764 )
32         return 0LL;
33     if ( 3645600 * a1[17] != 174988800 )
34         return 0LL;
35     if ( 10115280 * a1[16] != 981182160 )
36         return 0LL;
37     if ( 9667504 * a1[18] != 493042704 )
38         return 0LL;
39     if ( 5364450 * a1[19] != 257493600 )
40         return 0LL;
41     if ( 13464540 * a1[20] != 767478780 )
42         return 0LL;
43     if ( 5488432 * a1[21] != 312840624 )
44         return 0LL;

```

首先import, 然后声明变量的类型及名字, 然后创建solver(), 约束条件用add, 最后必须check(),打印
print (.model())

```
from z3 import *
```

```

s=Solver()
a1=[0]*32
for i in range(32):
    a1[i]=Int("a1["+str(i)+"]")
s.add( 1629056 * a1[0] == 166163712 )
s.add( 6771600 * a1[1] == 731332800 )
s.add( 3682944 * a1[2] == 357245568 )
s.add( 10431000 * a1[3] == 1074393000 )
s.add( 3977328 * a1[4] == 489211344 )
s.add( 5138336 * a1[5] == 518971936 )
s.add( 7532250 * a1[7] == 406741500 )
s.add( 5551632 * a1[8] == 294236496 )
s.add( 3409728 * a1[9] == 177305856 )
s.add( 13013670 * a1[10] == 650683500 )
s.add( 6088797 * a1[11] == 298351053 )
s.add( 7884663 * a1[12] == 386348487 )
s.add( 8944053 * a1[13] == 438258597 )
s.add( 5198490 * a1[14] == 249527520 )
s.add( 4544518 * a1[15] == 445362764 )
s.add( 3645600 * a1[17] == 174988800 )
s.add( 10115280 * a1[16] == 981182160 )
s.add( 9667504 * a1[18] == 493042704 )
s.add( 5364450 * a1[19] == 257493600 )
s.add( 13464540 * a1[20] == 767478780 )
s.add( 5488432 * a1[21] == 312840624 )
s.add( 14479500 * a1[22] == 1404511500 )
s.add( 6451830 * a1[23] == 316139670 )
s.add( 6252576 * a1[24] == 619005024 )
s.add( 7763364 * a1[25] == 372641472 )
s.add( 7327320 * a1[26] == 373693320 )
s.add( 8741520 * a1[27] == 498266640 )
s.add( 8871876 * a1[28] == 452465676 )
s.add( 4086720 * a1[29] == 208422720 )
s.add( 9374400 * a1[30] == 515592000 )
s.add(5759124 * a1[31] == 719890500)
s.check()
print(s.model())

```

```

a1 = [0]*32
a1[31] = 125
a1[30] = 55
a1[29] = 51
a1[28] = 51
a1[27] = 57
a1[26] = 51
a1[25] = 48
a1[24] = 99
a1[23] = 49
a1[22] = 97
a1[21] = 57
a1[20] = 57
a1[19] = 48
a1[18] = 51
a1[16] = 97
a1[17] = 48
a1[15] = 98

```

```
a1[14] = 48
a1[13] = 49
a1[12] = 49
a1[11] = 49
a1[10] = 50
a1[9] = 52
a1[8] = 53
a1[7] = 54
a1[5] = 101
a1[4] = 123
a1[3] = 103
a1[2] = 97
a1[1] = 108
a1[0] = 102
for i in range(32):
    if i == 6:
        continue
    print(chr(a1[i]), end="")
```

要注意的是z3变量不能用于普通的类型的式子中，因此最好全部把结果拿出来后再转换打印

不过这题有点问题，题目没给a1【6】，但是网上有大佬爆破出来是1

flag{e165421110ba03099a1c039337}